Parallel Neural Network Training with MPI: Comparing Gradient Aggregation Strategies

Jason Nair, Moustafa Kassem, Leo Lu

INTRODUCTION

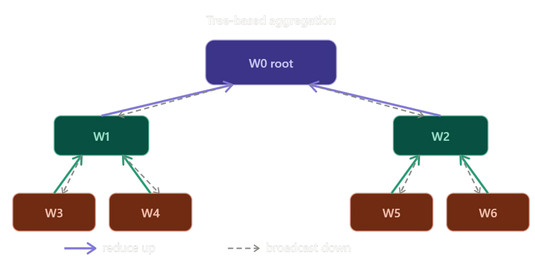
For our project, our aim is to implement a neural network with Stochastic Gradient Descent (SGD) and with data parallelism in C++ with MPI and OpenMP. The main goal is to compare the three different gradient aggregation strategies which are tree based, and ring all reduce, as well as halving-doubling. We want to see how each scales as we increase our worker count and model size. We aim to try overlapping gradient aggregation with our backward pass using non-blocking MPI calls, which will allow each of our layer's gradients to start communication as soon as they are ready instead of waiting for the whole back-prop iteration to finish. We think that ring all reduce should scale closer to the bandwidth optimal bound for larger gradient buffers, and the non-blocking version should also help to shave off additional time.

BACKGROUND

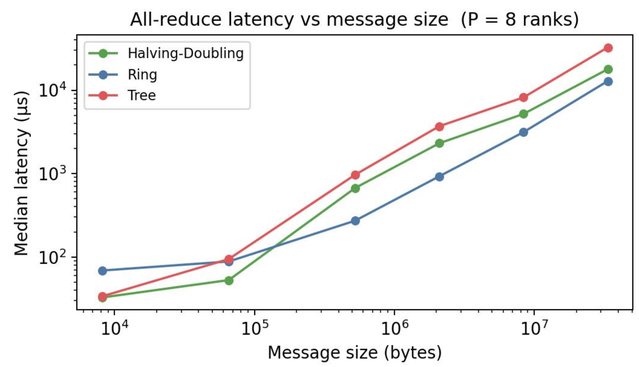
We implement this SGD synchronization with MPI, which uses explicit message passing between processes and scales across nodes, and with OpenMP, which shares memory between threads on a single node and avoids network communication entirely.
The synchronization itself is an all-reduce operation, and the algorithm used to perform it has a large effect on performance. Tree reduction finishes in a logarithmic number of steps and works well for small gradient buffers, but wastes bandwidth at scale. Ring all-reduce approaches the bandwidth-optimal bound for large buffers at the cost of more communication steps. Halving-doubling aims to combine the strengths of both. Our work builds on the Distributed Neural Net repository [1] as a starting point and uses PyTorch's Distributed Data Parallel [2] as a real-world performance reference.

METHODOLOGY

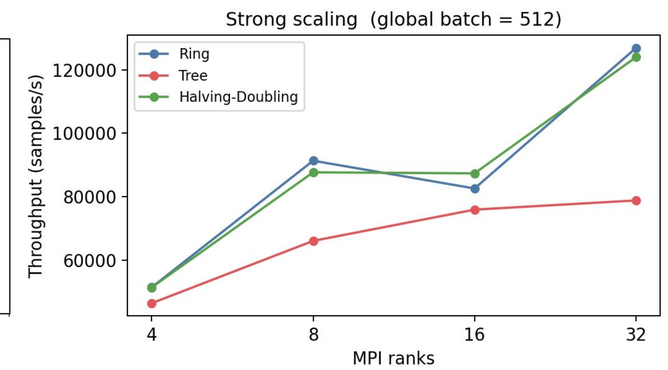
We build a configurable feedforward neural network in C++ with Eigen, training on MNIST (784→256→128→10, ~235K parameters). We test tree-based reduction, ring all-reduce, and recursive halving-doubling via strong scaling on Perlmutter CPU nodes with up to 32 MPI ranks, measuring iteration time, throughput, and communication cost. We vary network size across four configurations (50K–1.46M parameters) to show how gradient buffer length affects each algorithm differently. We use the α-β model to interpret the latency and bandwidth tradeoffs across the three strategies, and thread the forward and backward pass with OpenMP on a single node to compare intra-node parallelism against distributed MPI scaling.



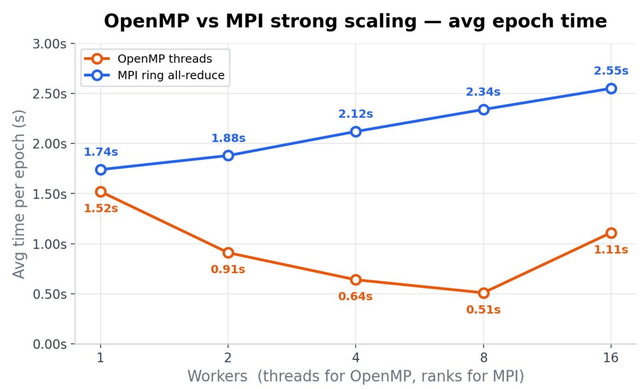
RESULTS



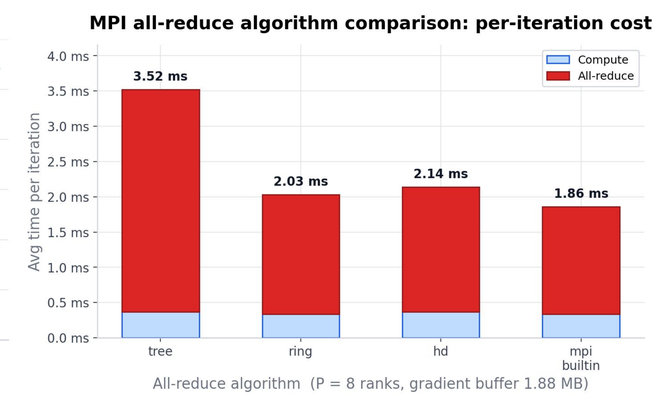
At small message sizes all three algorithms perform similarly, but ring's bandwidth-optimal design pulls ahead as gradient buffers grow. Tree wastes bandwidth by sending the full gradient at every reduction step, while ring only sends 1/P of the data per step.



Ring and halving-doubling scale consistently with more workers, while tree hits a communication bottleneck early. At 32 ranks ring achieves 60% higher throughput than tree, showing that algorithm choice matters more as parallelism increases.



OpenMP wins at this small model size, hitting 0.51 s/epoch at 8 threads (3x speedup) before NUMA boundaries hurt scaling. MPI's 1.88 MB all-reduce per iteration dominates the small per-rank compute at global batch 64.



Compute time is identical across algorithms; the difference lives entirely in the all-reduce phase. Tree pays a 2.N.log(P) bandwidth penalty. At P=8 that's a 3x volume cost vs ring's bandwidth-optimal 2.N.(P-1)/P.

FUTURE WORK

A definite goal going forward is to test the different strategies on larger neural networks or datasets. MNIST is a relatively small dataset, so efficiencies may become more apparent when data sizes become larger. We would also like to test on a multi-node slingshot.

A further optimization we investigate is non-blocking MPI communication: rather than waiting for the entire backward pass to finish before aggregating, each layer's gradients can begin transmitting as soon as they are ready, overlapping communication with computation.

In the future, we would also like to compare the efficiency of our implementations of data parallel SGD to Pytorch DDP as a reference point for an industry standard. This will help us understand whether the differences in performance are a result of the aggregation strategies or inefficiencies in the code structure.